

(graph) simple : no parallel arcs, no loops

Strassen's algo

$n = 2$ in only 7 multiplications (and 18 additions).

$$\begin{aligned}x_1 &= (a_{11} + a_{22}) * (b_{11} + b_{22}) & x_5 &= (a_{11} + a_{12}) * b_{22} \\x_2 &= (a_{21} + a_{22}) * b_{11} & x_6 &= (a_{21} - a_{11}) * (b_{11} + b_{12}) \\x_3 &= a_{11} * (b_{12} - b_{22}) & x_7 &= (a_{12} - a_{22}) * (b_{21} + b_{22}) \\x_4 &= a_{22} * (b_{21} - b_{11})\end{aligned}$$

$n = 2^k$ ($n \neq 2^k$: add 'dummy' row/cols)

$$\begin{aligned}A(0) &= 1 \\A(k) &= 7A(k-1) + 18(n/2)^2 \\&= 7 \cdot n^{\log 7} - 6n^2 \approx 7 \cdot n^{2.807} - 6n^2\end{aligned}$$

divide & conquer:

divide into smaller subprobs
solve each one recursively
combine to get result

worst $W(n)$ vs average $A(n)$ case
→ solve recurrence via expansion
optimal := best worst case

[binary tree]

$$\text{nodes} + \text{leaves} \leq 2^{d+1} - 1$$

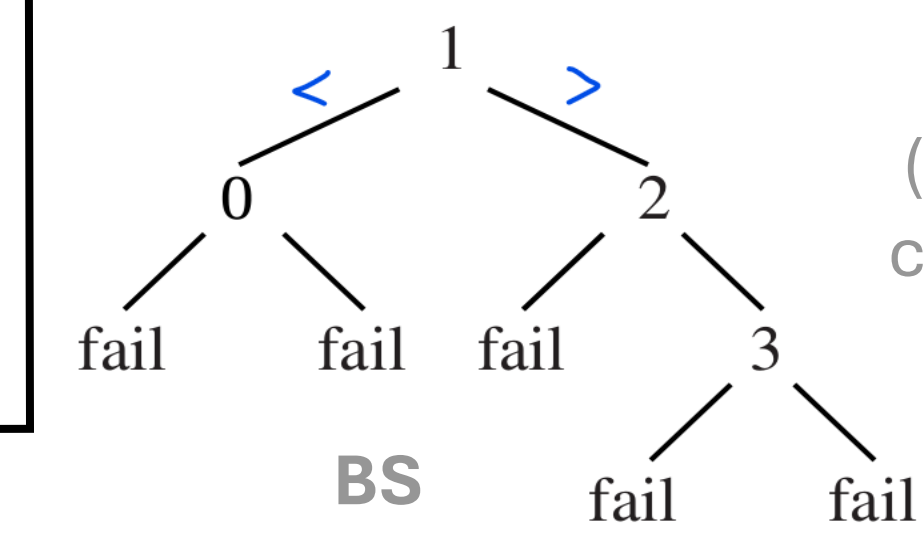
$$\text{leaves} \leq 2^d$$

LS: search unordered list

Modified LS: search ordered list

BS: search ordered from centre

DECISION TREES



MBS
()s indicate no comparison needed

```
i = 1
while i < n:
    # insert L[i] into L[0..i-1]
    j = i
    while L[j-1] > L[j] and j > 0:
        Swap(j-1, j)
        # swaps L[j-1] and L[j]. L[j]
        j = j-1
    i = i+1
```

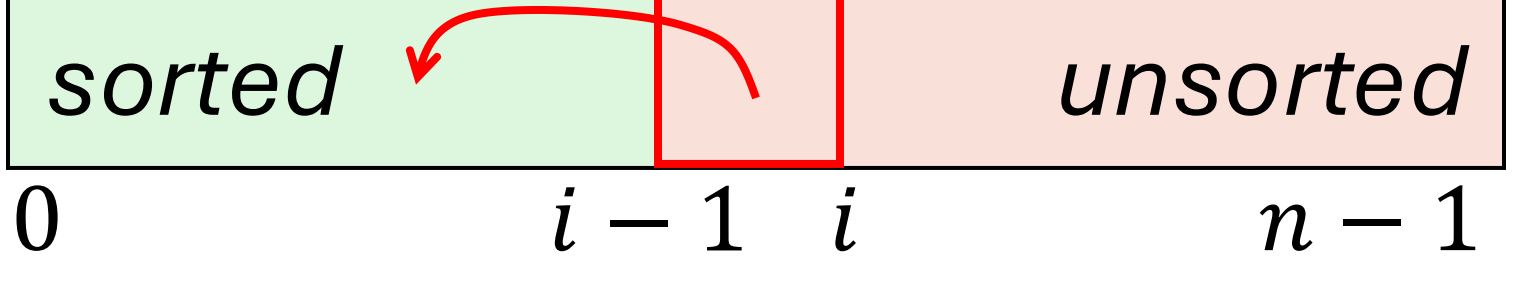
```
procedure Mergesort(left, right):
    if left < right:
        mid = ⌊(left + right)/2⌋
        # left ≤ mid < right
        Mergesort(left, mid)
        Mergesort(mid+1, right)
        Merge(left, mid, right)
```

```
procedure Quicksort(left, right):
    if left < right:
        [7, 2, 10, 12, 3, 1, 8]
        s = Split(left, right) [3, 2, 1, 7, 12, 8, 10]
        # left ≤ s ≤ right
        # L[s] is now in correct position
        Quicksort(left, s-1)
        Quicksort(s+1, right)
```

sorting worst case lower bound
Decision tree: node = comparison, leaf = result (sorted list) (⇒ find max depth)
 $n!$ perms (i.e. leaves) ⇒ $d \geq \lceil \log(n!) \rceil$
sorting average case LB
total path length := $\Sigma(\text{depth of each leaf})$
 $A(n) = \text{tpl}/(n!) \Rightarrow$ need lower bound on **tpl**
tpl minimal when tree balanced.
then every node depth is d or $d-1$
⇒ $A(n) \geq \lceil \log(n!) \rceil$

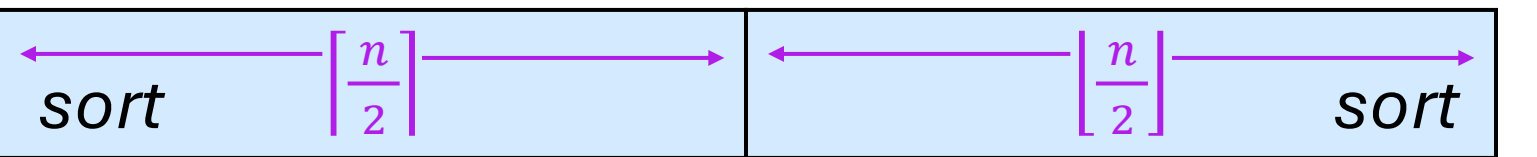
Insertion sort $\Theta(n^2)$

insert via successive swaps until in place
(worst case i comparisons: $W(n) = \frac{n(n-1)}{2}$)



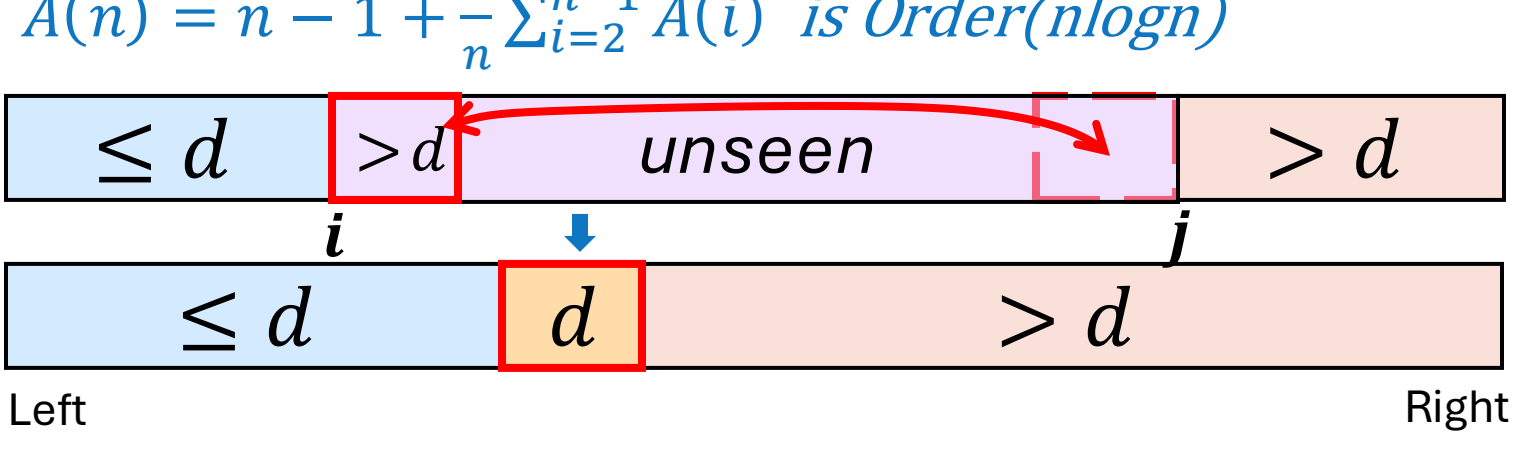
Mergesort $\Theta(\log(n!)) = \Theta(n \log n)$

split, sort each half, then merge.
worst case for merge (interleaving): $n-1$
(assume $n = 2^k$) $W(n) = n-1 + 2W(n/2)$
 $= n \log n - n + 1$



Quicksort $W(n) = \Theta(n^2)$ $A(n) = \Theta(n \log n)$

split list ar. 1st elem ($n-1$); recurse
(worst case alr. sorted: $W(n) = \frac{n(n-1)}{2}$)
 $A(n) = n-1 + \sum_{i=2}^n A(i)$ is Order($n \log n$)



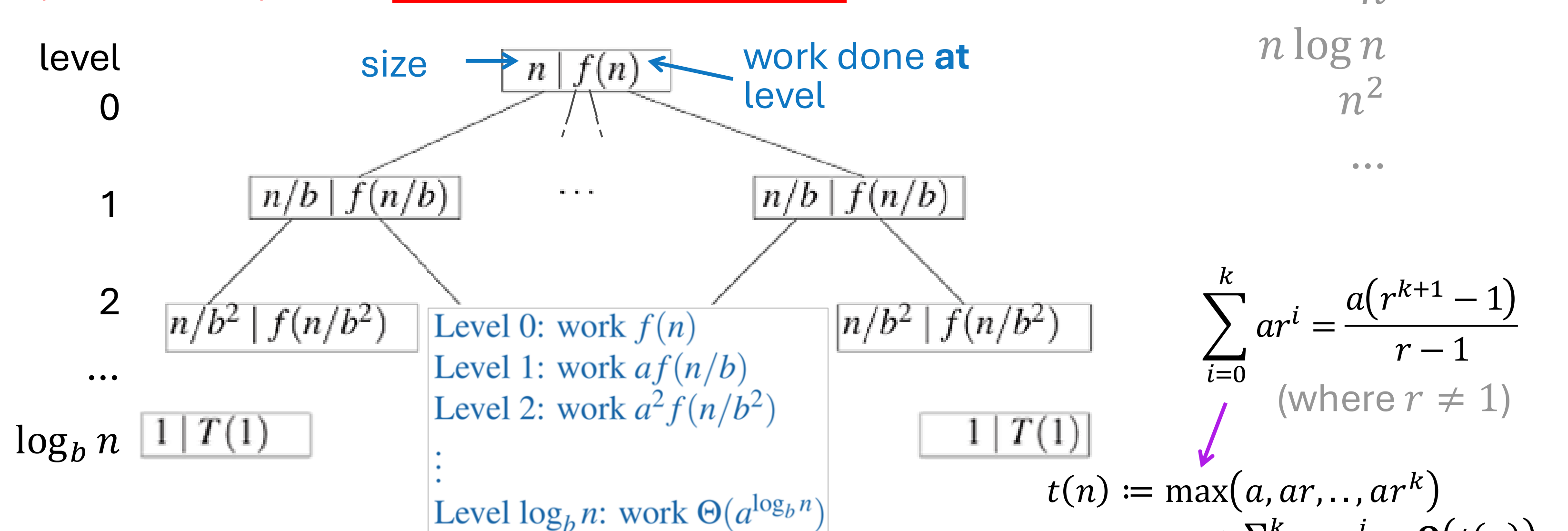
Order

$f, g: \mathbb{N} \rightarrow \mathbb{R}^+$
 f is $O(g) \Leftrightarrow \exists m \in \mathbb{N}, c \in \mathbb{R}^+$ s.t. $\forall n \geq m. [f(n) \leq c \cdot g(n)]$
 f is $\Theta(g) \Leftrightarrow f$ is $O(g)$ and g is $O(f)$

increasing order:
 $\log n = O(n^m)$ for all $m > 0$

Master Theorem (recursion tree)

$$T(n) = aT(n/b) + f(n)$$



critical exponent $E = \log_b a = \frac{\log a}{\log b}$

f is n^c general (where $\varepsilon > 0$)
 $E < c$ $n^{E+\varepsilon} = O(f(n)) \Rightarrow T(n) = \Theta(f(n))$ non-recursive work at root of recursion tree dominates
 $E = c$ $f(n) = \Theta(n^E) \Rightarrow T(n) = \Theta(f(n) \log n)$ work done at each level of recursion tree roughly equal
 $E > c$ $f(n) = O(n^{E-\varepsilon}) \Rightarrow T(n) = \Theta(n^E)$ recursive work at leaves of recursion tree dominates best f

$$T(n) = 2T(2n/3) + n \log n$$

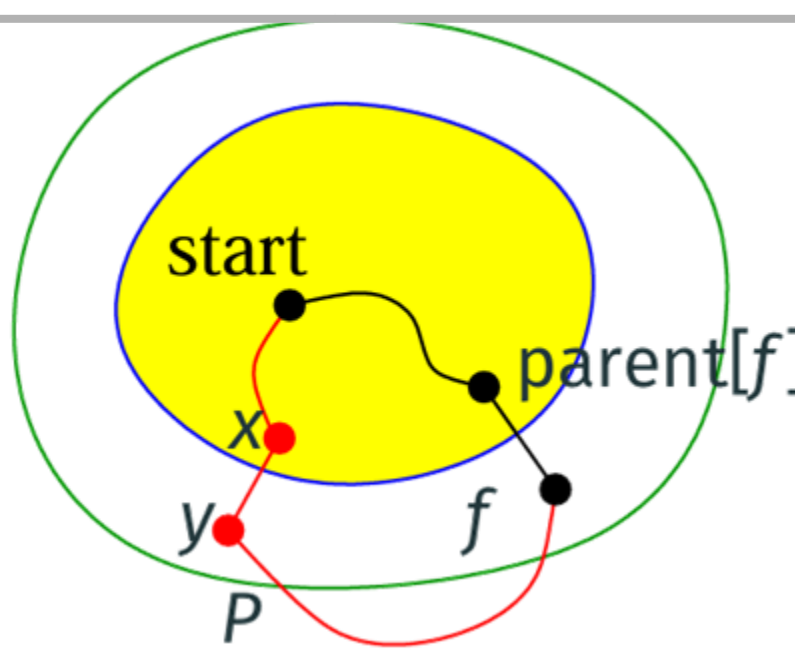
(c) Here $a = 2$ and $b = 3/2$ and $f(n) = \Theta(n \log n)$. So $E = \frac{\log a}{\log b} = \frac{\log 2}{\log 1.5} = 1.7095$. Set $\varepsilon = 0.5$.
So $f(n) = O(n^{E-\varepsilon})$ and solution is $\Theta(n^E) = \Theta(n^{1.7095})$. The recursive work at the leaves of the recursion tree dominates.

Proof that invariant (2) maintained (e.g.)

Let y be the first node on P not to belong to the tree.
By (3), $\text{distance}[y] \leq \text{distance from start to } y \text{ using } P$.
Hence length of $P \geq \text{distance}[y]$.

But $\text{distance}[y] \geq \text{distance}[f]$ by our choice of f .

Hence P is at least as long as $\text{path}(f)$.



bipartite : $\exists$ partition X, Y (of nodes) s.t. no arcs in X
no arcs in Y
⇒ 2-colourable
no adj nodes have same colour

Dijkstra $O(n^2)$

like Prim's but add x based on total dist from start
node is [tree/fringe/unseen]
gives shortest path tree
OR: stop when at finish (not MST)

A*

like Dijkstra, but compare fringe x on total dist $g(x) + h(x)$

$h(x)$ consistent $\triangleq$
• adj. $x, y \Rightarrow h(x) \leq W(x, y) + h(y)$
• $h(\text{finish}) = 0$ (underest. dist.)
 $\Rightarrow h(x) \leq \text{len}(P) + h(y)$ ($P = x \rightarrow y$)
 $h(x)$ admissible $\triangleq$
 $h(x) \leq W(x \rightarrow \text{finish})$ (shortest P)
consistent $\Rightarrow$ admissible

B_k is matrix with 0 or 1, B_k calculated from B_{k-1} dynamically

Warshall (transitive closure) $O(n^3)$

k th iteration (B_k): $\exists$ path $i \rightarrow j$ w/ only nodes $1..k$ allowed as intermediate
 $B_k[i, j] = B_{k-1}[i, j]$ or $(B_{k-1}[i, k] \text{ and } B_{k-1}[k, j])$
finish (and return) $B := B_n$

Floyd (shortest path btw all pairs) $O(n^3)$

like Warshall, but k th iter gives length of shortest path $i \rightarrow j$ (or ∞ if none)
 $B_k[i, j] = \min(B_{k-1}[i, j], B_{k-1}[i, k] + B_{k-1}[k, j])$

$B_k[i, j] = \min(B_{k-1}[i, j], B_{k-1}[i, k] + B_{k-1}[k, j])$

TSP = find minimal HC Bellman-Held-Karp $O(n^2 2^n)$

$C(S, x) = \min_{x \neq 1} \text{cost of path } 1 \rightarrow x \text{ for intermed. exactly } S$
soln $= \min_{x \neq 1} C(\text{nodes} \setminus \{1, x\}) + W(x, 1)$
 $C(S, x) = \min_{y \in S} C(S \setminus y) + W(y, x)$

Process sets S in incr. order of size.
for $S \in \text{Nodes} \setminus \{1\}$ with $S \neq \emptyset$:

Algorithm deleteMax(H)

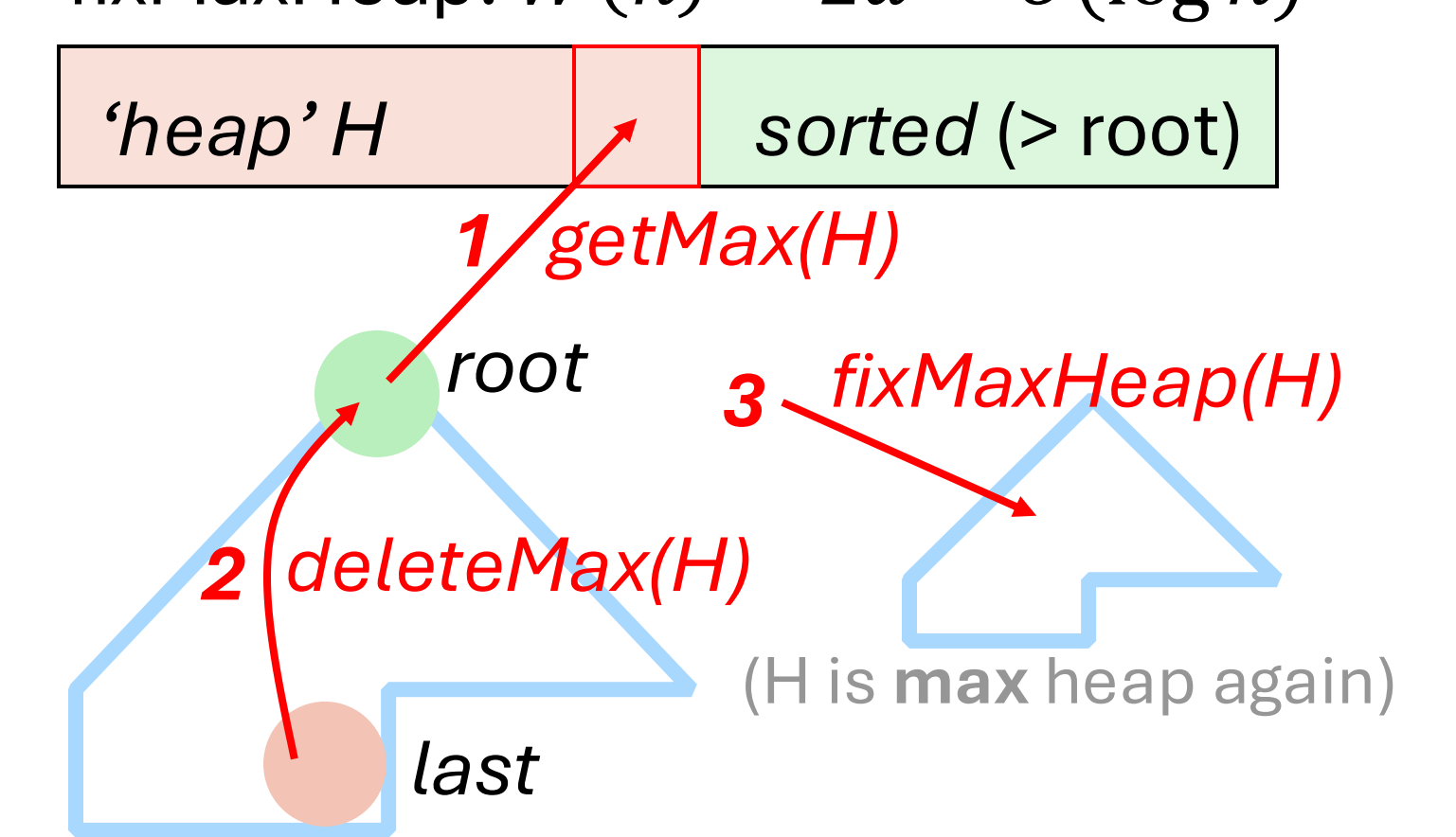
copy element at last node into root node
remove last node
fixMaxHeap(H)

Algorithm fixMaxHeap(H)

if H not a leaf:
largerSubHeap = the left or right subheap with the larger root
if $\text{root}(H).key < \text{root}(\text{largerSubHeap}).key$:
swap elements at $\text{root}(H)$ and $\text{root}(\text{largerSubHeap})$
fixMaxHeap(largerSubHeap)

Heapsort $O(n \log n)$

buildMaxHeap: heap struct $\rightarrow$ max heap
 $W(n) = 2W(\frac{n-1}{2}) + 2 \log n = \Theta(n)$ ($E=1$)
fixMaxHeap: $W(\frac{n}{2}) = 2d = O(\log n)$



Algorithm Heapsort scheme:

Build max heap H out of an array E of elements $O(n)$ for $i = n$ to 1:
max = getMax(H) $O(1)$
deleteMax(H) $O(\log n)$
 $E[i] = \text{max}$

Algorithm buildMaxHeap(H)

if H not a leaf:
buildMaxHeap(left subtree of H)
buildMaxHeap(right subtree of H)
fixMaxHeap(H)

Heap

Heap structure $\triangleq$ left-complete binary tree

$\triangleq$ all nodes are present @ depth $0..d-1$

depth d : no nodes missing on left of a present node

T is 'maximising partial order tree' $\triangleq$ node $\geq$ all children

max heap $\triangleq$ heap structure & maximising partial order tree

min heap $\triangleq$ similar (node $\leq$ all children instead)

T is 'minimising partial order tree' $\triangleq$ node $\leq$ all children

as an array: start @ index 1

node @ $i \Rightarrow$ left child @ $2i$, right child @ $2i+1$

$\Rightarrow$ parent @ $\lfloor i/2 \rfloor$



Heap can be a PQ:

deleteMin(Q):
 $E[1] = E[\text{heapsize}]$
decrement heapsize
fixMinHeap(1, heapsize)

percolateup(c):

if $c > 1$:
parent = $\lfloor c/2 \rfloor$
if $E[c].key < E[\text{parent}].key$:
swap(c, parent)
percolateup(parent)

insert(Q, x):
heapsize = heapsize+1
 $E[\text{heapsize}] = x$
allow x to percolate towards the root until heap property restored
percolateup(heapsize)

Invariant proving correctness of Dijkstra

- If x is a tree or fringe node (other than start) then $\text{parent}[x]$ is a tree node.
- If x is a tree node (other than start) then $\text{distance}[x]$ is the length of shortest path, and $\text{parent}[x]$ is its predecessor along that path.
- If f is a fringe node then $\text{distance}[f]$ is the length of the shortest path where all nodes except f are tree nodes. Furthermore, $\text{parent}[f]$ is its predecessor along that path.

classic priority queue

"nodes added in order: a,h,g,f,b,c,d,e"

PRIM

Min PQ:

- each $x \in Q$ has priority $\text{key}[x]$
 - items removed lowest key first
- operations:
- ```
Q = PQcreate()
isEmpty(Q)
insert(Q, x)
getMin(Q)
deleteMin(Q)
decreaseKey(Q, x, newkey)
```

(in both: while (??) non-empty: (fringe nodes / arcs left)

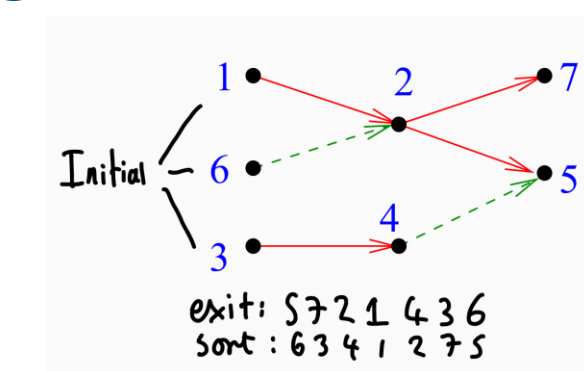
## KRUSKAL

```
procedure dfs(x):
 visited[x] = true
 print x
 for y in adj[x]:
 if not visited[y]:
 parent[y] = x
 dfs(y)
 # backtrack to x
 else if not exited[y]:
 abort # cycle
 exited[x] = true
 ts[i] = x; i--
```

```
visited[x] = true # x root
print x
enqueue(x, Q)
while not isEmpty(Q):
 y = front(Q)
 for z in adj[y]:
 if not visited[z]:
 visited[z] = true
 print z
 parent[z] = y
 enqueue(z, Q)
 dequeue(Q)
```

- connected? (DFS or BFS): list of visited nodes (from start)
- has cycle? (DFS, or BFS): return arc connecting two ancestors:

```
for y in adj[x]:
 if visited[y] and y != parent[x]:
 return (x, y) # cycle found
 (...)
```



- distances from start (BFS): map of distance  
replace print z with  $\text{distance}[z] = \text{distance}[y] + 1$

- Topological sort (DFS on a DAG): array w/ reverse of exit order  
total ordering:  $x_1, \dots, x_n$  s.t.  $i > j \Rightarrow \neg \exists \text{ path } x_i \rightarrow x_j$

size = # nodes

**Decision Problem** ( $D$ ; also  $A$ )  
 $D$  **decided by**  $A \triangleq A$  always terminates, returns 'yes' or 'no' depending on  $D(x)$   
 $D$  **decidable in p-time**  $\triangleq D$  decided by  $A$ , which runs in p-time (of input size [polynomial  $p$ ] for all inputs of size  $n$ ,  $A$  takes  $\leq p(n)$  steps  
problem tractable  $\triangleq$  worst case is polynomial time

Polynomial invariance thesis  
If a problem can be solved in p-time in some reasonable model of computation, then it can be solved in p-time in any other reasonable model of computation.

**P-time**  
problem can be efficiently solved  
 $D(x) \in P \triangleq$  decidable in  $p(n)$  time in some reasonable MOC, ( $n$  = input size  $|x|$ )  
arithmetical operations (+, -, x, ÷) are p-time: polynomial in input size  $\log n$   
 $f, g$  are p-time computable  $\Rightarrow g \circ f$  is p-time computable

**NP** (non-deterministic polynomial time)  
problem can be efficiently verified  
 $D(x) \in NP \triangleq$  exists decision problem  $E(x, y) \in P$  and polynomial  $p(n)$  s.t.  
•  $D(x) \Leftrightarrow \exists y. E(x, y)$  ( $E$  verifies  $D$ )  
•  $E(x, y) \Rightarrow |y| \leq p(|x|)$  " $E$  is poly balanced"  
otherwise it would take too long to guess  $y$

$E$  is a verification problem

**Reduction**  
 $D \leq D'$  ( $D$  reduces to  $D'$ )  $\triangleq$  exists p-time-computable  $f$  s.t.  $D(x) \Leftrightarrow D'(f(x))$   
reduce a question about  $D$  (easier) to a question about  $D'$  (harder)  
 $D \leq D'$  and  $D' \in P \Rightarrow D \in P$   
 $D \leq D'$  and  $D' \in NP \Rightarrow D \in NP$

**NP-hard**  
 $D$  is NP-hard  $\triangleq \forall D' \in NP, D' \leq D$  (NB:  $D$  not necessarily in NP)  
 $D$  is NP-complete  $\triangleq D \in NP$  and  $D$  is NP-hard  
SAT ("is this CNF formula satisfiable?") is NP-complete ( $x+y$ ).( $a+b$ )  
HamPath ("does this graph contain an HP") is NP-complete

**P vs NP**  
 $P \subseteq NP$ , we do not know if the reverse holds  
assuming  $P \neq NP$ :  
 $D$  NP-hard  $\Rightarrow D' \notin P$

**Travelling salesman problem (TSP)**  
Decision problem: given (complete) weighted graph  $(G, W)$  and bound  $B$ , is there a tour of  $G$  w/ total weight  $\leq B$ ?  
tour  $\triangleq$  every node once & return to start (i.e. HC)

**TSP(D) is NPC:** To show TSP is intractable, define a decision problem and show that it is NPC

If we guess a path  $p$ , we can check in p-time that  $p$  is a Hamiltonian circuit of  $G$  and that  $W(p) \leq B$ . Clearly  $|p| \leq |G|$ .

More formally define  $VER\text{-}TSP(D)((G, W), B, p)$  iff  $p$  is a Hamiltonian circuit of  $(G, W)$  and  $W(p) \leq B$ .

Then  $TSP(D)((G, W), B)$  iff  $\exists p. VER\text{-}TSP(D)((G, W), B, p)$ .

Also if  $VER\text{-}TSP(D)((G, W), B, p)$  then  $|p| \leq |G|$  under reasonable definitions of size.

$D' \leq TSP(D)$  for some  $D' \in NPC$   
We choose HAMPATH as the known NPC problem and show  $HAMPATH \leq TSP(D)$ .

We need to define a p-time function  $f$  which transforms a graph  $G$  into a weighted graph  $(G', W)$  together with a bound  $B$  so that  $HAMPATH(G)$  iff  $TSP(D)((G', W), B)$ .

Given  $G$  we construct  $(G', W)$  as follows: Set nodes( $G'$ ) = nodes( $G$ ). Given any two distinct nodes  $x, y$  of  $G$ :

- If  $(x, y)$  is an arc of  $G$  then  $(x, y)$  is also an arc of  $G'$ , with  $W(x, y) = 1$ .
- If  $(x, y)$  is not an arc of  $G$  then  $(x, y)$  is an arc of  $G'$ , with  $W(x, y) = 2$ .

Thus we add in the missing arcs of  $G$  but we give them a higher weight. Finally we let  $B = n + 1$  where  $G$  has  $n$  nodes.

It is not hard to see that  $f(G) = ((G', W), B)$  is p-time — easiest to see using the adjacency matrix representation.

We now check that  $f$  is a reduction:  $HAMPATH(G)$  iff  $TSP(D)((G', W), B)$ : Suppose  $G$  has a Hamiltonian path  $\pi$  with endpoints  $x$  and  $y$ . The same path in  $G'$  has weight  $n - 1$  (all arcs have weight 1). We get a Travelling Salesman tour by adding in arc  $(x, y)$  with  $W(x, y) \leq 2$ . Thus we have a tour of weight  $\leq n + 1 = B$ .

Conversely, suppose  $(G', W)$  has a tour of weight  $\leq B = n + 1$ . This has  $n$  arcs. So at most one arc can have weight 2. Suppose this arc has endpoints  $x, y$ . Then omitting arc  $(x, y)$  gives us a Hamiltonian path in  $G$ .

**MSTs – how many / unique?** (\*) (which node is 'missing' from  $T'$ )  
• All STs have the same number of arcs ( $n-1$ ), which is 6. Call the MST obtained  $T$ .  
• Any other ST would have to use (at least) one of the arcs not used by  $T$ , and avoid using (at least) one of the arcs in  $T$   
• All arcs of weight  $\leq 3$  (AD, EF, ED, BD) are already in  $T$ , so if any ST omits one of these arcs, it must have replaced it with a more costly arc, making the resulting ST more costly and so not an MST. Thus any MST must include these arcs.  
• Including these arcs in some candidate MST  $T'$ , (\*) the only arc in  $T$  not yet in  $T'$  is AC, which has weight 4. for  $T'$  to be a distinct MST, it must therefore replace AC in  $T$  with some other arc of weight 4. The only other such arcs are AB and BE.  
• replacing AC w/ BE creates a cycle (BED), so this is not a tree and so not an MST.  
• replacing AC w/ AB creates a spanning tree, and the overall weight has not changed (AC and AB have equal weight). So this does create an MST.  
• Thus there are two possible MSTs, due to the choice between arcs AB and AC

**Prim: assumptions**  
(assuming that we choose numerically least when there are two choices- e.g. between 5 and 7).  
**CAREFUL:**  
If C is added to fringe w/ weight 4 via AC, say, and then B is added to fringe, we don't update C's parent to B unless BC is STRICTLY less than AC  
"algo only changes parent if it offers an advantage"

**NB: discussing colourings**  
If  $f(G)$  has a 4-colouring  $c$ , then  $c(x) \neq c(y)$  for all  $y \in \text{nodes}(G)$ . So  $c$  restricted to nodes( $G$ ) is a 3-colouring of  $G$ .

**Algorithm Prim's algorithm with priority queues:**  
Choose any node start as the root  
tree[start] = true  
for  $x$  in adj[start]:  
key[x] =  $\infty$ ;  
parent[x] = nil  
insert( $Q, x$ )  
decreaseKey( $Q, \text{start}, 0$ )  
while not isEmpty( $Q$ ):  
f = getMin( $Q$ ); deleteMin( $Q$ )  
tree[f] = true  
for  $y$  in adj[f]:  
if not tree[y]: # so  $y$  in  $Q$   
if  $W[f, y] < \text{key}[y]$ :  
decreaseKey( $Q, y, W[f, y]$ )  
parent[y] =  $f$

If sparse:  $m \leq n \log n$ , PQ + Kruskal better  
If dense: Classic prim better

**ISOMORPHISM NP: GUESS A 1-1 MAPPING FROM T-M  $O(n^2)$**   
**PATHS NP: GUESS A PATH + ADD CHECKS**

**Show HAMCYCLE is in NP**

We guess a sequence of nodes  $\pi(x_1, \dots, x_n)$  and then check in p-time that it is a Hamiltonian cycle for  $G$ .

Define the verification problem is  $VER\text{-}HAMCYCLE(G, \pi)$ :

- $G$  and  $\pi$  has  $n$  nodes
- all  $x_i$  are distinct and nodes of  $G$
- there is an arc in  $G$  between  $x_i$  and  $x_{i+1}$  ( $i = 1, \dots, n$ ) (including one between  $x_n$  and  $x_1$ )

Then (a) and (b) are clearly  $O(n)$ , and (c) is  $O(m) = O(n^2)$  as it requires iterating through the arcs in  $G$  (for  $n$  nodes,  $m$  arcs), so  $VER\text{-}HAMCYCLE(G, \pi) \in P$ , and  $|\pi|$  is bounded in  $|G|$  so p-balanced.

Also we have  $HAMCYCLE(G)$  iff  $\exists \pi. VER\text{-}HAMCYCLE(G, \pi)$

**Show HAMCYCLE is NP-Complete**

We show  $HAMPATH \leq HAMCYCLE$

Let  $G$  have  $n$  nodes. Define a reduction function  $f(G) = G'$  such that  $f$  adds a new node  $x$  to  $G$  and joins every node of  $G$  to  $x$  by  $n$  new arcs. This construction is p-time, as we add p-many nodes (only one) and arcs ( $n$ ).

- Suppose  $G$  has a HP with endpoints  $y, z$ . Then  $f(G)$  has a HC where we add the arcs  $(y, x)$  and  $(x, z)$ .
- Conversely, suppose that  $f(G)$  has a HC. WLOG, we can suppose that this starts at  $x$  and is of the form  $x, y, \dots, z, x$ . We can omit node  $x$  and get  $y, \dots, z$ , which is a HP for  $G$ . We have shown  $HAMPATH \leq HAMCYCLE$ .

So, since  $HAMPATH$  is NP-hard, **HAMCYCLE is also NP-hard. Thus, as HAMCYCLE is in NP, it is NP-complete.**

**Quicksort: Split**

**Algorithm Split(left, right):**  
# pre-condition: left < right  
 $d = L[\text{left}]$  # pivot  
 $i = \text{left} + 1$   
 $j = \text{right}$   
# Invariant:  
# left <  $i \leq j$  + 1  
#  $j \leq \text{right}$   
# if left <  $k < i$  then  $L[k] \leq d$   
# if  $j < k \leq \text{right}$  then  $L[k] > d$   
while  $i \leq j$ :  
if  $L[i] \leq d$ :  
 $i = i + 1$   
else:  
Swap( $i, j$ )  
 $j = j - 1$   
#  $i = j + 1$   
Swap(left,  $j$ )  
return  $j$  # index of splitpoint

Let  $G$  be simple with  $n$  nodes, and suppose every node of  $G$  has degree  $\geq n/2$ . Show  $G$  connected.  
Assume not connected. Let  $C$  be any connected component of  $G$ . Since  $G$  is simple, any node of  $C$  must be connected to  $\geq n/2$  other nodes, all of which are in  $C$ . Hence  $|C| \geq n/2 + 1$ , there are multiple components but then  $G$  cannot have  $\geq 2$  connected components, as that would mean that  $G$  has  $\geq 2(n/2 + 1) = n + 2$  nodes, which is a contradiction.  
Hence  $G$  is connected.  
minimum number of swaps to sort  $[n-1, 0, 1, \dots, n-2]$  ( $n \geq 2$ )  
Can be done in  $n-1$  swaps by swapping the  $n-1$  with each succeeding element to get it to the end of the list. We see that all items except the  $n-1$  need to move one position to the left. But a single swap can only move one item leftwards. Hence  $n-1$  swaps are needed.  
[this is key: only one 'loser' / one 'winner' per comparison]

**# comparisons in worst case (via decision tree)**

The decision tree must have 5 leaves (for the 5 different results). A tree of depth  $\leq 2$  can have at most 4 leaves. Hence the tree must have depth at least 3, giving a lower bound of 3 comparisons in the worst case.

**Algorithm Find with path compression:**  
procedure cfnd( $x$ ):  
 $y = \text{parent}[x]$   
if  $y == x$ : #  $x$  is the root  
root =  $x$   
else:  
root = cfnd( $y$ )  
if root !=  $y$ :  
parent[x] = root  
return root

**PROOF ADVICE:**

- Maximal path technique ( $d=1$  in tree, sink in DAG)
- Pigeonhole + PBC
- Induction

HPA is defined as follows: given an undirected (simple) graph  $G$  and an arc  $a$  joining nodes  $x$  and  $y$  of  $G$ , is there a Hamiltonian path in  $G$  which uses  $a$ ?

Show HPA is NP-complete (assuming HP NP-complete)

To show HPA is NP-hard, we show that  $HP \leq HPA$ :

Let  $G$  have  $n$  nodes. Let  $f(G) = (G', a)$  where  $G'$  has 2 new nodes  $x, y$ , with  $x$  joined to every node of  $G$ , and with  $y$  joined to  $x$  by arc  $a$ . Then  $f$  is clearly p-time: we add p-many nodes (two) and arcs ( $n+1$ ).

- Suppose that  $G$  has an HP  $x_1, \dots, x_n$ . Then  $f(G)$  has an HP  $y, x, x_1, \dots, x_n$  using  $a$ .
- Conversely, suppose that  $f(G)$  has an HP using  $a$ . Clearly  $y$  must be an endpoint since it has degree one. The HP must be of the form  $y, x, x_1, \dots, x_n$ . Then  $x_1, \dots, x_n$  is an HP of  $G$ .

As HPA is NP-hard and in the complexity class NP (not shown), it is NP-complete

**ACTUAL HeapSort algos**

**fixMaxHeap(root, heapsize):**  
left =  $2 * \text{root}$   
right =  $2 * \text{root} + 1$   
if left  $\leq$  heapsize:  
# root is not a leaf  
if left = heapsize:  
# no right subheap  
largerSubHeap = left  
elif  $E[\text{left}].\text{key} > E[\text{right}].\text{key}$ :  
# favours right subheap if equal  
largerSubHeap = left  
else:  
largerSubHeap = right  
if  $E[\text{root}].\text{key} < E[\text{largerSubHeap}].\text{key}$ :  
swap( $\text{root}$ , largerSubHeap)  
fixMaxHeap(largerSubHeap, heapsize)

**# HeapSort on elements 1..n of array E**  
heapsize =  $n$   
buildMaxHeap(1, heapsize)  
# Invariant:  
# max heap in  $E[1..heapsize]$  of first heapsize-many elements of sorted list  
# elements in heapsize+1 to  $n$  correctly sorted  
while heapsize > 1:  
swap(1, heapsize)  
heapsize = heapsize - 1  
fixMaxHeap(1, heapsize)

**buildMaxHeap(root, heapsize):**  
left =  $2 * \text{root}$   
right =  $2 * \text{root} + 1$   
if left  $\leq$  heapsize:  
# root is not a leaf  
buildMaxHeap(left, heapsize)  
if right  $\leq$  heapsize:  
# there is a right subtree  
buildMaxHeap(right, heapsize)  
fixMaxHeap(root, heapsize)

5. [From the 1999 exam] A sorted list of length  $2n$  is disarranged to create a new list  $L$  by placing the elements in the odd positions (still in order) before the elements in even positions (still in order). Thus e.g. the list  $[1, 2, 3, 4, 5, 6]$  would become  $[1, 3, 5, 2, 4, 6]$ .  
How many comparisons does Insertion Sort take when applied to  $L$ ? Explain your working.

5. The list can be represented as  $[1, 3, \dots, 2n-1, 2, 4, \dots, 2n]$ . The list  $[1, 3, \dots, 2n-1]$  is already sorted. So each insertion takes one comparison, making  $n-1$  comparisons in all. When inserting the element  $2i$  ( $i = 1, \dots, n$ ), the list looks like  $[1, 2, \dots, 2i-1, 2i+1, \dots, 2n-1, 2i, 2i+2, \dots, 2n]$ . So  $2i$  has to be compared with  $2n-1, 2n-3, \dots, 2i-1$  ( $n+1-i$  comparisons). Hence total is

$$n-1 + \sum_{i=1}^n (n+1-i) = n-1 + \frac{n(n+1)}{2}$$

7. Suppose that Mergesort is to be applied to a list with  $n$  elements (where  $n$  is a power of two), which is already sorted, but in reverse order.

(a) Write down a recurrence relation for  $R(n)$ , the number of comparisons required, explaining your answer briefly.

The list is split into two portions of length  $n/2$  which are sorted separately, taking  $R(n/2)$  comparisons each. Every element of the LH portion  $A$  is greater than every element of the RH portion  $B$ . So the merge will compare each element of  $B$  with the least element of  $A$ , until  $B$  is exhausted. This takes  $n/2$  comparisons. At the end of this  $A$  can be written in without further comparisons.

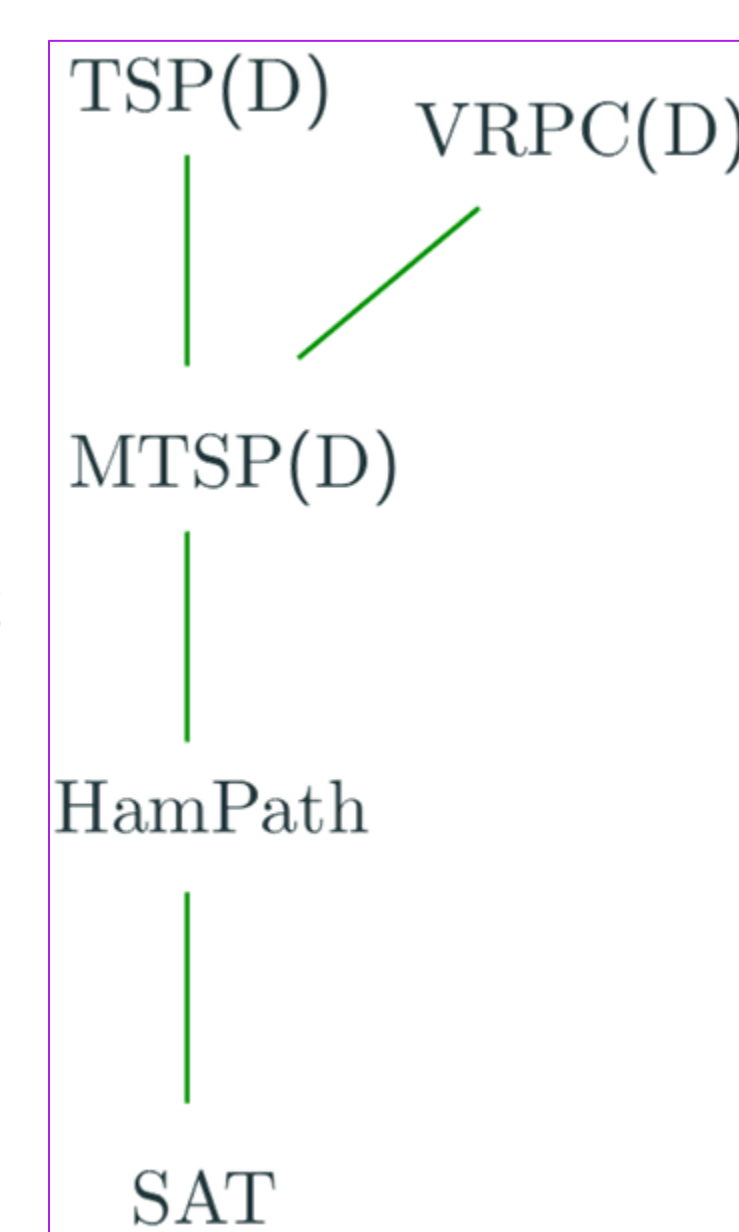
Show VRPC is NP-complete (assuming HP NP-complete)

Guess an assignment of packages to specific vehicles and itinerary for each vehicle. Verify:

- Total size of package on each vehicle doesn't exceed  $C$
- Itinerary for each vehicle starts at depot, visits all assigned delivery addresses and returns to depot
- Total combined cost of all routes does not exceed budget  $B$
- Every package assigned to exactly one vehicle

(1) is  $O(k)$ , (2) is  $O(n)$  and (3) is  $O(n^2)$  so verifiable in p-time. Since assignments and itinerary bound by  $|G|$ , it's p-balanced, so NP.

Suppose that  $(G, W)$  has  $n$  nodes. Then  $f((G, W), B) = ((G', W'), B', x)$  where  $(G', W') = (G, W)$  and  $B = B'$ . Choose one node to be the depot ( $x$ ) and assign 1 package size 1 to be delivered to the rest of the nodes and 1 vehicle with capacity  $n-1$ .



**Vehicle routing problem with capacities VRPC(D)**  
Given  
• a complete weighted graph  $(G, W)$  satisfying the triangle inequality,  
• a distinguished node start (the depot),  
•  $k$  vehicles with capacity  $C$ ,  
• a set of packages with sizes  $s_1, \dots, s_n$  to be delivered to nodes  $x_1, \dots, x_n$  respectively,  
• and a budget  $B$ :

can the packages be delivered within total cost  $B$ , subject to total size of the packages on each vehicle being within the capacity  $C$ ?

**Proven in problem sheets:**

- (undirected) 2 colourable  $\leftrightarrow$  no odd length cycles
- 2-colourable, all nodes same degree  $\rightarrow$  for any 2-col, same # nodes each colour
- (simple) all degrees  $\geq n/2 \rightarrow$  connected
- (connected)  $n \geq 3 \rightarrow$  at least one node degree  $\geq 2$
- tree w/ 2 nodes has at least one node of degree 1
- (simple) number of arcs  $\leq n(n-1)/2$
- ( $n \geq 2$ ) every node degree at least 3  $\rightarrow$  has cycle of length  $\leq 2\lceil \log n \rceil$
- arc added to a ST  $\rightarrow$  now has cycle  $C$  containing  $a$
- removing any other arc from  $C$  creates a new ST
- no two arcs have same weight  $\rightarrow$  unique MST

**Algorithm Floyd's algorithm:**  
input  $A$   
 $\text{set } B[i, j] = \begin{cases} 0 & \text{if } i = j \\ A[i, j] & \text{if } i \neq j \text{ and there is an arc } (i, j) \\ \infty & \text{otherwise} \end{cases}$   
#  $B = B_0$   
for  $k = 1$  to  $n$ :  
for  $i = 1$  to  $n$ :  
for  $j = 1$  to  $n$ :  
 $b_{ij} = \min(b_{ij}, b_{ik} + b_{kj})$   
return  $B$